AI-Agentic SOC Setup

A step-by-step installation & configuration guide for deploying PostgreSQL and automated threat triage workflows.

Phase 1: Project Setup

Creating project skeleton, version control, and python environment insulation.

Step 1: Project Folder Structure

Directory Architecture

Establish the root project directory and subdirectories as specified in the technical design document to compartmentalize modules cleanly.

Subdirectories: agent, apis, database, frontend, docs, tests

Execution Command

Run the shell command to provision the entire tree structure instantaneously:

```
mkdir -p ai-agentic-soc/{agent,apis,database,frontend,
docs,tests}
cd ai-agentic-soc
```

Step 2: Git & Python Environment

- **Initialize Git Repository:** Track changes from inception with a clean ignore profile for cache and virtual environments.

```
git init && echo "__pycache__/,.env,venv/" > .gitignore && git add . && git commit -m "Initial commit: project structure"
```

- **Create Python Virtual Environment:** Isolate dependencies for environment stability.

```
# Linux / macOS: python3 -m venv venv && source venv/bin/activate
# Windows (PowerShell): python -m venv venv && .\venv\Scripts\Activate
```

Step 3: Core Package Installation

Tech Stack Components

Install core orchestration packages for agent graphs, language model hooks, database connectivity, and the operator dashboard UI.

Includes LangGraph, LangChain, Anthropic SDK, FastAPI, Uvicorn, Streamlit, Pydantic, and Psycopg2.

Pip Execution

```
pip install --upgrade pip
pip install langgraph langchain anthropic fastapi uvic
orn streamlit psycopg2-binary pydantic requests
```

Phase 2: Database & Telemetry

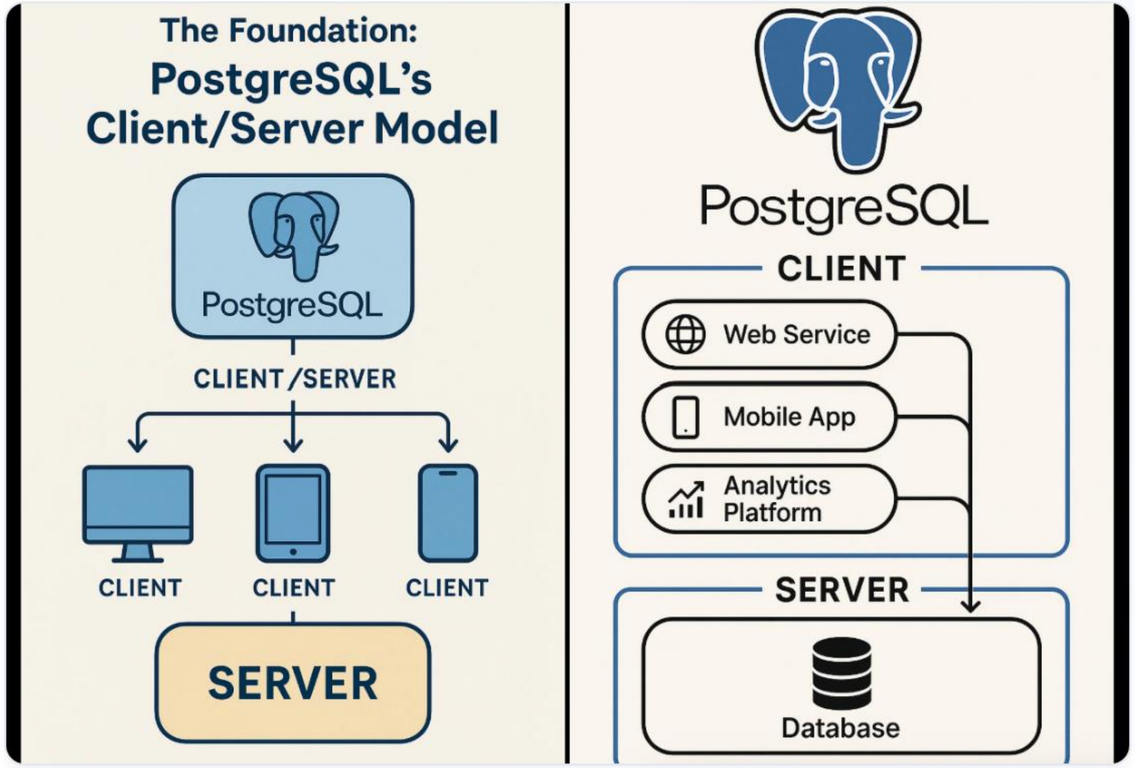
PostgreSQL persistence layer setup and SIEM telemetry integration.

Step 4: PostgreSQL Configuration

PostgreSQL stores investigation states, reasoning-chain logs, and test evaluation results.

```
# Start Service:
# Ubuntu: sudo systemctl start postgresql
# macOS: brew services start postgresql

# Create Database & User:
sudo -u postgres psql
CREATE DATABASE soc_triage_db;
CREATE USER soc_admin WITH ENCRYPTED PASSWORD 'secure_passwor
d_here';
GRANT ALL PRIVILEGES ON DATABASE soc_triage_db TO soc_admin;
```



Step 4: Verify Database Connection

Verification Script (verify_db.py)

Create a Python script inside your database folder to test connectivity and print the server version.

```
import os, psycopg2
try:
    conn = psycopg2.connect(database="soc_triage_db", user="soc
_admin", password="secure_password_here", host="localhost")
    cur = conn.cursor()
    cur.execute("SELECT version();")
    print("Connected:", cur.fetchone()[0])
    conn.close()
except Exception as e:
    print("Error:", e)
```

Execution

Run the verification script from your root directory:

```
python database/verify_db.py
```

Expected output confirms successful handshake and authentication with PostgreSQL.

Step 5: Wazuh SIEM Telemetry

Architecture & Execution Flow:

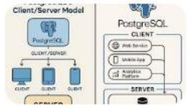
- **Wazuh Agents:** Deployed on endpoints (Windows/Linux) feeding telemetry & logs (Sysmon/FIM).
- **FastAPI Ingestion:** Ingests JSON-formatted security alerts from Wazuh Manager.
- **LangGraph Agent:** Dynamically plans investigation tools (EDR, historical SIEM queries).
- **Analyst Report:** Compiles MITRE ATT&CK-mapped timeline and risk assessment.

Ready for Deployment

Your AI-Agentic SOC environment is fully initialized, structured, and connected.

Questions or Further Integrations?

Image Sources



https://miro.medium.com/v2/resize:fit:1400/1*k8VeoT5phtgwKDuAJVCa_A.png

Source: medium.com

https://www.progressive.in/assets/img/photos/SOC-Dashboard.webp?format=webp

Source: www.progressive.in